

P1 - Simple Mail Client

9/18/2026

100 Points Possible

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

9/20/2026

Details

Week 4 · 100 points · about 8 hours · submit in Canvas

Overview

You are going to write a mail client that talks to an SMTP server directly, over a socket, speaking the protocol by hand. No mail library. The whole point is that once you have read [RFC 5321 \(https://www.rfc-editor.org/rfc/rfc5321\)](https://www.rfc-editor.org/rfc/rfc5321) you can implement an application layer protocol from its specification, which is what the rest of this course assumes you can do.

Fork the [starter repository \(https://github.com/shanep/makefile-project-starter\)](https://github.com/shanep/makefile-project-starter) using the **Use this template** button and name your copy `cs425-p1`.

Warning

Your code must compile on **GitHub Codespaces** and **Onyx**. If it compiles on only one of them you will receive a zero even if it works on the other.

Learning Outcomes

- 2.1 Write a client that speaks a text-based protocol over a socket
- 2.2 Explain the client-server application architecture
- 7.1 Build code with a build system and run it under a unit test framework
- 7.2 Produce code free of memory leaks and out-of-bounds accesses

Background

SMTP is a text protocol. A client connects to port 25, the server greets it with a `220`, and from there the two sides exchange lines of ASCII. Every server reply starts with a three digit status code, and the client's job is to check that code before it sends the next command.

A minimal session looks like this, with `C:` for what your client sends and `S:` for what the server sends back:

```
S: 220 smtp.example.com ESMTP ready
C: HELO onyx.boisestate.edu
S: 250 smtp.example.com
C: MAIL FROM:<you@boisestate.edu>
S: 250 2.1.0 Ok
C: RCPT TO:<someone@example.com>
S: 250 2.1.5 Ok
C: DATA
S: 354 End data with .
C: Subject: hello
C:
C: This is the message body.
C: .
S: 250 2.0.0 Ok: queued
C: QUIT
S: 221 Bye
```

So the codes your client requires, in order, are **220, 250, 250, 250, 354, 250, 221**. Anything else at that point in the session is an error.

Three details bite people every year:

- Lines are terminated with **CRLF**, `\r\n`, not `\n`.
- The message body ends with a line containing a single period. A body line that legitimately starts with a period has to be escaped by doubling it. Section 4.5.2 of the RFC calls this out; miss it and a message silently truncates instead of failing loudly.
- **A reply is not always one line.** A server may answer with a continuation, where every line but the last has a hyphen after the code instead of a space:

```
S: 250-smtp.example.com
S: 250-PIPELINING
S: 250 SIZE 10240000
```

Your client has to read the whole reply before it decides anything.

Speak `HELO`, not `EHLO`. You are not negotiating any ESMTP extension, and there is no TLS and no authentication in this project.

The test server

Do not point this at a campus mail relay or at a real mail provider. There is a test server for the class as shown below.

```
instance  i-099fcec43f9463ff8 (running, t3.micro)
host      ec2-54-148-3-55.us-west-2.compute.amazonaws.com
ip        54.148.3.55
ports     25 587 2525
web       http://ec2-54-148-3-55.us-west-2.compute.amazonaws.com/
mailbox   /var/spool/cs425-mail/mailbox
```

It is a sink. It speaks a real RFC 5321 session and accepts mail for any recipient, then writes the message to a file and stops. Nothing is delivered and nothing is relayed, so you can send whatever you like to it as often as you like.

It listens on **25**, **587** and **2525**. Campus networks and home ISPs very often block outbound port 25, so if your connection just hangs, that is almost certainly what happened: use `-p 2525` instead. That is a property of the network you are sitting on, not a bug in your client.

Task 1 - Complete the program

Your program is built as `./build/release/myapp` and must take this command line:

```
Usage: myapp -f <from> -t <to> [-s subject] [-b body] [-p port]
        [-H hello-host] <server>
```

```
-f <from>      envelope sender, for example you@example.com
-t <to>        envelope recipient
-s <subject>    subject line (default: empty)
-b <body>      message body (default: read from stdin)
-p <port>      port or service name (default: 25)
-H <hello-host> host name sent with HELO (default: localhost)
<server>      host name or address of the mail server
```

```
echo "This is the message body." | \
./build/release/myapp -f me@boisestate.edu -t you@example.com \
-s "hello" -H onyx.boisestate.edu <server posted in Canvas>
```

Use `getopt` for the parsing. The interface is fixed because it is what is exercised when your project is graded, so match it exactly.

The client must:

- Open a TCP connection to the server named on the command line, on the given port. Resolve the name with `getaddrinfo`; do not assume it is a dotted quad.
- Read the greeting and verify the status code before sending anything.
- Send `HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, the message and `QUIT`, checking the server's reply at every step and following continuation lines.
- Build the message itself as `From`, `To` and `Subject` headers, a blank line, the dot stuffed body, and the `.` line that ends it.

- Fail cleanly with a useful error message if any reply is not what the protocol expects. Do not crash, and do not carry on after an error. Say which reply the server actually sent; "an error occurred" helps nobody.
- Exit **0** when the server queues the message, **1** when the command line is wrong and **2** when the connection or the SMTP session fails.
- Print the usage message and exit **0** when run with no arguments at all. This is what `make leak` runs, so it has to be a clean, successful path.

Tip

Reject an address or a subject containing a bare CR or LF rather than sending it. Passing one through lets whoever supplied it inject an extra SMTP command or mail header into your session, which is the SMTP version of an injection bug.

Task 2 - Design for testability

You cannot unit test against a live mail server, so the protocol logic must not touch a socket. This separation is most of the design work in this project and it is graded. Split `src/lab.h` into three layers:

1. **Pure protocol helpers.** Functions that take strings and return strings or status codes: parse a reply line into its code, decide whether a reply line is the final one, build a command line, dot stuff a body, build the `DATA` payload. No I/O anywhere in here.
2. **The session, over a transport you can swap out.** Reading a line, reading a whole reply, writing, sending one command and checking its code, and running the entire session. These do all their reading and writing through a pair of function pointers (a `read` and a `write` callback plus a context pointer) rather than calling `recv` and `send` directly.
3. **The socket transport.** Thin wrappers over `getaddrinfo`, `connect`, `recv` and `send` that satisfy those two callbacks.

The payoff is layer 2. The real client plugs a socket into it, and your tests plug in a scripted in-memory server, which means you can drive a complete session and every one of its error paths with no network at all. A scripted server is just a string, and an error case is that string truncated or with one code changed.

Be aware that reading a line is not one `recv`. A reply can arrive split across several reads, or several replies can arrive in one, so your reader needs a buffer that it refills only when it does not already hold a complete line.

Task 3 - Testing

Add Unity tests for every function you declare in `src/lab.h`.

```
make check
```

Beyond the happy path, make sure you have tests for a multi-line reply, a reply that arrives a few bytes at a time, a reply the buffer cannot hold, a server that hangs up in the middle of the session, and each wrong status code in the sequence.

Task 4 - Coverage

```
make clean
make all
make report
```

Fix your tests until you have 100% coverage with everything passing. As in P0, you may only exclude branches originating from system or library calls.

Task 5 - Leak and crash check

- Run `make leak`, then `make leak-test`.
- Fix every leak and every crash. A client that leaks on the error path still leaks, and the error paths are exactly where the allocations get missed.

Task 6 - Replace README.md

Task 7 - Continuous integration

- Push everything to GitHub and confirm the CI run is green.
- Run the **Create Submission Report Via GitHub Action** workflow.
- Download `submission-report.docx` once it completes.

Submitting

1. Download `submission-report.docx` from GitHub and submit it to Canvas.
2. Check your submission in Canvas so you know it arrived intact.

Rubric		
#	Criterion	Points
1	Client takes the required command line, connects to the named server and checks the greeting	10
2	The full SMTP command sequence is issued in the correct order	20
3	Every reply is checked, continuation lines are followed, and a bad status code is reported rather than ignored	20
4	CRLF line endings, dot stuffing and end-of-data handling are correct	10
5	Protocol logic is separated from socket I/O behind a swappable transport	10
6	Unity tests cover every function in <code>lab.h</code> with 100% coverage	15
7	No memory leaks or crashes under <code>make leak-test</code> , including on error paths	10
8	<code>README.md</code> replaced and CI green on the last push	5

View Rubric

P1 - Simple Mail Client rubric (1)			
Criteria	Ratings		Points
Client takes the required command line, connects to the named server and checks the greeting	Full marks	No marks	/10 pts
	10 pts	0 pts	
The full SMTP command sequence is issued in the correct order	Full marks	No marks	/20 pts
	20 pts	0 pts	
Every reply is checked, continuation lines are followed, and a bad status code is reported rather than ignored	Full marks	No marks	/20 pts
	20 pts	0 pts	
CRLF line endings, dot stuffing and end-of-data handling are correct	Full marks	No marks	/10 pts
	10 pts	0 pts	
Protocol logic is separated from socket I/O behind a swappable transport	Full marks	No marks	/10 pts
	10 pts	0 pts	

Criteria	Ratings		Points
Unity tests cover every function in `lab.h` with 100% coverage	Full marks 15 pts	No marks 0 pts	/15 pts
No memory leaks or crashes under `make leak-test`, including on error paths	Full marks 10 pts	No marks 0 pts	/10 pts
`README.md` replaced and CI green on the last push	Full marks 5 pts	No marks 0 pts	/5 pts

Choose a submission type

Upload

⋮

More



Choose a file to upload

Webcam Photo

Canvas Files

Submit

Assess